

Sprawozdanie z Laboratorium

Systemy Wbudowane

Autor: Krystian Dobrołęcki 175008

Spis treści

1	Opisy środowiska pracy, sprzętu i oprogramowania	2
2	Zadanie 1: Program przełączający cyklicznie 6 podprogramów	2
2.1	Krótki opis zadania	2
2.2	Kod realizujący zadanie	2
3	Zadanie 2: Alarm oparty na potencjometrze	5
3.1	Krótki opis zadania	5
3.2	Kod realizujący zadanie	5
4	Zadanie 3: Reklama świetlna na LCD i LED	7
4.1	Krótki opis zadania	7
4.2	Kod realizujący zadanie	7
5	Zadanie 4: Kontroler kuchenki mikrofalowej	9
5.1	Krótki opis zadania	9
5.2	Kod realizujący zadanie	9
6	Zadanie 5: Zegar szachowy	12
6.1	Krótki opis zadania	12
6.2	Kod realizujący zadanie	12
7	Problemy i trudności	15
8	Źródła	15

1 Opisy środowiska pracy, sprzętu i oprogramowania

Realizacja wszystkich zadań odbyła się w oparciu o zestaw **Explorer 16/32** wyposażony w mikrokontroler **PIC24FJ128GA010**.

Do implementacji oraz wgrywania oprogramowania wykorzystano środowisko **MPLAB X IDE**. Kod został napisany w języku C, przy użyciu kompilatora **XC16**. Do wykonania zadań wykorzystano wbudowane na płycie przyciski (S3, S4, S5, S6), potencjometr, diody LED oraz wyświetlacz LCD.

2 Zadanie 1: Program przełączający cyklicznie 6 podprogramów

2.1 Krótki opis zadania

Celem zadania jest stworzenie maszyny stanów, która przy użyciu przycisków pozwala na przełączanie się między sześcioma różnymi licznikami (binarne, Graya, BCD - zliczające w górę i w dół). Nazwa aktualnego programu ukazywana jest na wyświetlaczu LCD, a działanie samego licznika pokazują diody led na płycie.

2.2 Kod realizujący zadanie

Poniżej zaprezentowano kod realizujący zadanie 1.

Listing 1: Program przełączający cyklicznie 6 podprogramów

```
1 #include <xc.h>
2 #include <libpic30.h>
3 #include <stdbool.h>
4 #include <stdint.h>
5 #include "buttons.h"
6 #include "lcd.h"
7
8 //8-bitowy licznik binarny w górę
9 int prog1(void) {
10     unsigned char count = 0;
11
12     LCD_ClearScreen();
13     LCD_PutString("P1: BIN W GORE", 14);
14
15     // Zabezpieczenie drgań styków - czekamy na puszczenie przycisków
16     while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
17         __delay32(10000); }
18
19     while(1) {
20         if (BUTTON_IsPressed(BUTTON_S6)) return 2;
21         if (BUTTON_IsPressed(BUTTON_S3)) return 6;6
22
23         LATA = count;
24         count++;
25
26         __delay32(2000000);
27     }
28 }
```

```

29 //8-bitowy licznik binarny w dół
30 int prog2(void) {
31     unsigned char count = 255;
32
33     LCD_ClearScreen();
34     LCD_PutString("P2: BIN W DOL", 13);
35
36     while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
37         __delay32(10000); }
38
39     while(1) {
40         if (BUTTON_IsPressed(BUTTON_S6)) return 3;
41         if (BUTTON_IsPressed(BUTTON_S3)) return 1;
42
43         LATA = count;
44         count--; // Dekrementacja
45
46         __delay32(2000000);
47     }
48
49 //8-bitowy licznik kod Graya w górę
50 int prog3(void) {
51     unsigned char count = 0;
52
53     LCD_ClearScreen();
54     LCD_PutString("P3: GRAY W GORE", 15);
55
56     while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
57         __delay32(10000); }
58
59     while(1) {
60         if (BUTTON_IsPressed(BUTTON_S6)) return 4;
61         if (BUTTON_IsPressed(BUTTON_S3)) return 2;
62
63         //Konwersja kodu binarnego na kod Graya:
64         unsigned char grayValue = count ^ (count >> 1);
65         LATA = grayValue;
66
67         count++;
68         __delay32(2000000);
69     }
70
71 //8-bitowy licznik kod Graya w dół
72 int prog4(void) {
73     unsigned char count = 255;
74
75     LCD_ClearScreen();
76     LCD_PutString("P4: GRAY W DOL", 14);
77
78     while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
79         __delay32(10000); }
80
81     while(1) {
82         if (BUTTON_IsPressed(BUTTON_S6)) return 5;
83         if (BUTTON_IsPressed(BUTTON_S3)) return 3;

```

```

84         unsigned char grayValue = count ^ (count >> 1);
85         LATA = grayValue;
86
87         count--;
88         __delay32(2000000);
89     }
90 }
91
92 //2x4 bitowy licznik w kodzie BCD w górę (0...99)
93 int prog5(void) {
94     char tens = 0; // dziesiątki
95     char units = 0; // jedności
96
97     LCD_ClearScreen();
98     LCD_PutString("P5: BCD W GORE", 14);
99
100    while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
101        __delay32(10000); }
102
103    while(1) {
104        if (BUTTON_IsPressed(BUTTON_S6)) return 6;
105        if (BUTTON_IsPressed(BUTTON_S3)) return 4;
106
107        //Składanie dwóch cyfr na jednym porcie
108        LATA = (tens << 4) | units;
109
110        units++;
111        if (units > 9) {
112            units = 0;
113            tens++;
114            if (tens > 9) {
115                tens = 0; //wyzerowanie po dojściu do 99
116            }
117        }
118        __delay32(2000000);
119    }
120 }
121
122 //2x4 bitowy licznik w kodzie BCD w dół (99...0)
123 int prog6(void) {
124     char tens = 9;
125     char units = 9;
126
127     LCD_ClearScreen();
128     LCD_PutString("P6: BCD W DOL", 13);
129
130    while(BUTTON_IsPressed(BUTTON_S6) || BUTTON_IsPressed(BUTTON_S3)) {
131        __delay32(10000); }
132
133    while(1) {
134        if (BUTTON_IsPressed(BUTTON_S6)) return 1;
135        if (BUTTON_IsPressed(BUTTON_S3)) return 5;
136
137        LATA = (tens << 4) | units;
138
139        units--;
140        if (units < 0) {
141            units = 9;

```

```

140         tens--;
141         if (tens < 0) {
142             tens = 9;
143         }
144     }
145     __delay32(2000000);
146 }
147 }
148
149
150 int main(void){
151     //conf
152     AD1PCFG = 0xFFFF;
153     TRISA = 0x0000;
154
155     LCD_Initialize();
156
157     //start od programu nr 1
158     int active_program = 1;
159
160     //glowna petla sterujaca przełączaniem programow
161     while(1) {
162         switch(active_program) {
163             case 1: active_program = prog1(); break;
164             case 2: active_program = prog2(); break;
165             case 3: active_program = prog3(); break;
166             case 4: active_program = prog4(); break;
167             case 5: active_program = prog5(); break;
168             case 6: active_program = prog6(); break;
169             default: active_program = 1; break;
170         }
171     }
172
173     return 0;
174 }

```

3 Zadanie 2: Alarm oparty na potencjometrze

3.1 Krótki opis zadania

Zadanie polega na wykorzystaniu potencjometru jako progu. Jeśli potencjometr przekracza połowę zakresu, alarm włącza się (przez pierwsze 5 sekund mruga jedna dioda, następnie świecą się wszystkie). Alarm można wyłączyć przyciskiem (S3), o ile wartość potencjometru spadła poniżej progu 50%.

3.2 Kod realizujący zadanie

Listing 2: Alarm oparty na potencjometrze

```

1 // PIC24FJ128GA010 Configuration Bit Settings
2 // CONFIG2
3 #pragma config POSCMOD = XT
4 #pragma config OSCIOFNC = ON
5 #pragma config FCKSM = CSDCMD

```

```

6  #pragma config FNOSC = PRI
7  #pragma config IESO = ON
8  // CONFIG1
9  #pragma config WDTPS = PS32768
10 #pragma config FWPSA = PR128
11 #pragma config WINDIS = ON
12 #pragma config FWDTEN = OFF
13 #pragma config ICS = PGx2
14 #pragma config GWRP = OFF
15 #pragma config GCP = OFF
16 #pragma config JTAGEN = OFF
17
18 //taktowanie dla delay_32
19 #define FCY 4000000UL
20
21 #include <xc.h>
22 #include <libpic30.h>
23 #include <stdbool.h>
24 #include <stdint.h>
25 #include "buttons.h"
26 #include "adc.h"
27
28 typedef enum {
29     STAN_NORMALNY,
30     STAN_MRUGANIE,
31     STAN_CIAGLY
32 } StanAlarmu;
33
34 int main(void) {
35     AD1PCFG = 0xFFFF;
36     TRISA = 0x0000;
37     LATA = 0x0000;
38
39     ADC_SetConfiguration(ADC_CONFIGURATION_DEFAULT);
40     ADC_ChannelEnable(ADC_CHANNEL_POTENTIOMETER);
41
42     StanAlarmu obecnyStan = STAN_NORMALNY;
43     unsigned int wartoscPotencjometru = 0;
44     int licznikCzasu = 0;
45
46
47     //glowna petla programu
48     while (1) {
49         wartoscPotencjometru = ADC_Read10bit(ADC_CHANNEL_POTENTIOMETER);
50         if (wartoscPotencjometru == 0xFFFF) {
51             continue;
52         }
53
54
55         //jesli jest normalnie i przekroczymy 50% odpala sie alarm
56         if (obecnyStan == STAN_NORMALNY) {
57             if (wartoscPotencjometru >= 512) {
58                 obecnyStan = STAN_MRUGANIE;
59                 licznikCzasu = 0;
60             }
61         }
62         //jesli alarm trwa

```

```

63     else if (obecnyStan == STAN_MRUGANIE || obecnyStan ==
        STAN_CIAGLY) {
64
65         //proba wyłączenia dziala tylko gdy suwak <50%
66         if (wartoscPotencjometru < 512 && BUTTON_IsPressed(BUTTON_S3)
            == true) {
67             obecnyStan = STAN_NORMALNY;
68             LATA = 0x0000; // Zgaś diody
69         }
70
71         //jesli nie wyłączono nadal mruga
72         if (obecnyStan == STAN_MRUGANIE) {
73             if ((licznikCzasu / 5) % 2 == 0) {
74                 LATA = 0x0001;
75             } else {
76                 LATA = 0x0000;
77             }
78
79             licznikCzasu++;
80
81             //po 5sekundach ledy przechodza w tryb ciagly
82             if (licznikCzasu >= 50) {
83                 obecnyStan = STAN_CIAGLY;
84             }
85         }
86         else if (obecnyStan == STAN_CIAGLY) {
87             LATA = 0x00FF; //wszystkie diody sie swieca
88         }
89     }
90
91     __delay32(400000);
92 }
93
94 return 0;
95 }

```

4 Zadanie 3: Reklama świetlna na LCD i LED

4.1 Krótki opis zadania

Zadanie obejmuje symulację dynamicznej tablicy reklamowej. Do stworzenia efektu przesuwanego tekstu na wyświetlaczu LCD wykorzystałem bufor znaków oraz funkcje biblioteki `string.h` (m.in. `strncpy`). Dodatkowo dla efektu dodałem animowane diody LED.

4.2 Kod realizujący zadanie

Listing 3: Reklama na LCD i LED

```

1 #include <xc.h>
2 #include <libpic30.h>
3 #include <stdbool.h>
4 #include <stdint.h>
5 #include <string.h>
6 #include "buttons.h"
7 #include "lcd.h"

```



```

8
9  int main(void) {
10      AD1PCFG = 0xFFFF;
11      TRISA = 0x0000;
12      LATA = 0x0000;
13
14      LCD_Initialize();
15      LCD_ClearScreen();
16
17      //tekst do animacji
18      char scrollMsg[] = "          Nie wychodz po zmroku... Ucieczka jest
19                          niemożliwa... Obejrzyj STAMTAD NA NETFLIX!!!          ";
20      //obliczanie dlugosci komunikatu
21      int msgLen = strlen(scrollMsg);
22
23      while (1) {
24
25          for (int i = 0; i < 4; i++) {
26              LCD_ClearScreen();
27              LCD_PutString("  OSTRZEZENIE!  ", 16);
28              LATA = 0xFFFF;
29              __delay32(2000000);
30
31              LCD_ClearScreen();
32              LATA = 0x0000;
33              __delay32(1000000);
34          }
35
36          LCD_ClearScreen();
37          LCD_PutString("  Zostan w domu.  ", 16);
38          LATA = 0x0081;
39          __delay32(10000000);
40
41          for (int i = 0; i <= msgLen - 16; i++) {
42              LCD_ClearScreen();
43
44              char buffer[17];
45              strncpy(buffer, &scrollMsg[i], 16);
46              buffer[16] = '\0';
47
48              LCD_PutString(buffer, 16);
49
50              //migoczące diody
51              if (i % 2 == 0) {
52                  LATA = 0x00AA;
53              } else {
54                  LATA = 0x0055;
55              }
56
57              __delay32(1000000);
58          }
59
60          LATA = 0x0000;
61          __delay32(10000000); //pauza przed powtorka reklamy
62
63      }
64
65      return 0;
66  }

```

5 Zadanie 4: Kontroler kuchenki mikrofalowej

5.1 Krótki opis zadania

Zadanie polega na zaprojektowaniu prostego sterownika czasu i mocy kuchenki mikrofalowej wraz z maszyną stanów (GOTOWA, GRZANIE, PAUZA, KONIEC). Potencjometr odpowiada za regulację mocy wyrażaną w procentach. Przyciski służą do ustawiania czasu (S3), startu/pauzy (S4) oraz resetu (S6).

5.2 Kod realizujący zadanie

Listing 4: Kontroler kuchenki mikrofalowej

```
1 #include <xc.h>
2 #include <libpic30.h>
3 #include <stdbool.h>
4 #include <stdint.h>
5 #include <stdio.h>
6 #include "buttons.h"
7 #include "lcd.h"
8 #include "adc.h"
9
10 //stany pracy mikrofal
11 typedef enum {
12     STAN_GOTOWA,
13     STAN_GRZANIE,
14     STAN_PAUZA,
15     STAN_KONIEC
16 } StanMikrofali;
17
18 int main(void) {
19     AD1PCFG = 0xFFFF;
20     TRISA = 0x0080;
21     LATA = 0x0000;
22
23     ADC_SetConfiguration(ADC_CONFIGURATION_DEFAULT);
24     ADC_ChannelEnable(ADC_CHANNEL_POTENTIOMETER);
25
26     LCD_Initialize();
27     LCD_ClearScreen();
28
29     StanMikrofali stan = STAN_GOTOWA;
30     int czasSekundy = 0;           //czas grzania
31     int mocProcent = 0;           //moc w %
32     int odliczanieBazy = 0;       //licznik pętli do uciekania sekund
33
34     //zmienne do wykrywania wcisnięcia przycisku
35     bool s3Poprzednio = false;
36     bool s4Poprzednio = false;
37     bool s6Poprzednio = false;
38
39     //zmienne do odswiezania lcd
40     StanMikrofali staryStan = STAN_KONIEC;
41     int staryCzas = -1;
42     int staraMoc = -1;
43     char buforLcd[33];
44 }
```

```

45 while(1) {
46     //Odczyt mocy mikrofalni (0-100%)
47     unsigned int adcVal = ADC_Read10bit(ADC_CHANNEL_POTENTIOMETER);
48     if(adcVal != 0xFFFF) {
49         //poprawka: '100L' zapobiega bledom integeroverflow
50         mocProcent = (adcVal * 100L) / 1023;
51     }
52
53     bool s3Teraz = BUTTON_IsPressed(BUTTON_S3);
54     bool s4Teraz = BUTTON_IsPressed(BUTTON_S4);
55     bool s6Teraz = BUTTON_IsPressed(BUTTON_S6);
56
57     bool wcisnietoDodajCzas = s3Teraz && !s3Poprzednio; //+10 sekund
58     bool wcisnietoStartStop = s4Teraz && !s4Poprzednio; //start/
59         pauza
60         bool wcisnietoReset      = s6Teraz && !s6Poprzednio; //anuluj/
61         reset
62
63     s3Poprzednio = s3Teraz;
64     s4Poprzednio = s4Teraz;
65     s6Poprzednio = s6Teraz;
66
67     //przycisk S3 +10s:
68     if (wcisnietoDodajCzas) {
69         czasSekundy += 10;
70         if (czasSekundy > 3599) czasSekundy = 3599;
71
72         if (stan == STAN_KONIEC) {
73             stan = STAN_GOTOWA;
74             LATA = 0x0000;
75         }
76     }
77
78     //przycisk S6 reset:
79     if (wcisnietoReset) {
80         czasSekundy = 0;
81         stan = STAN_GOTOWA;
82         LATA = 0x0000;
83     }
84
85     //przycisk S4 start/stop
86     if (wcisnietoStartStop) {
87         if (stan == STAN_GOTOWA && czasSekundy > 0) {
88             stan = STAN_GRZANIE;
89         } else if (stan == STAN_GRZANIE) {
90             stan = STAN_PAUZA;
91             LATA = 0x0000;
92         } else if (stan == STAN_PAUZA && czasSekundy > 0) {
93             stan = STAN_GRZANIE;
94         } else if (stan == STAN_KONIEC) {
95             stan = STAN_GOTOWA;
96         }
97     }
98
99     //upływ czasu
100    if (stan == STAN_GRZANIE) {
        odliczanieBazy++;
    }

```

```

101 //animacja diod
102 int fazaAnimacji = (odliczanieBazy / 2) % 4;
103 LATA = (1 << fazaAnimacji);
104
105 //co 10 obiegów odlicz 1 sekundę
106 if (odliczanieBazy >= 10) {
107     odliczanieBazy = 0;
108     czasSekundy--;
109
110     if (czasSekundy <= 0) {
111         czasSekundy = 0;
112         stan = STAN_KONIEC;
113         LATA = 0x0000;
114     }
115 }
116
117 }
118 else if (stan == STAN_KONIEC) {
119     //gotowe - mruganie diodami
120     odliczanieBazy++;
121     if ((odliczanieBazy / 5) % 2 == 0) LATA = 0x00FF;
122     else LATA = 0x0000;
123 }
124 else {
125     odliczanieBazy = 0;
126 }
127
128 //wyswietlacz lcd
129 if (stan != staryStan || czasSekundy != staryCzas || (mocProcent
130 /2) != (staraMoc/2)) {
131     staryStan = stan;
132     staryCzas = czasSekundy;
133     staraMoc = mocProcent;
134
135     int minuty = czasSekundy / 60;
136     int sekundy = czasSekundy % 60;
137
138     char nazwaStanu[11];
139     if (stan == STAN_GOTOWA) sprintf(nazwaStanu, "GOTOWA");
140     else if (stan == STAN_GRZANIE) sprintf(nazwaStanu, "GRZANIE");
141     else if (stan == STAN_PAUZA) sprintf(nazwaStanu, "PAUZA");
142     else if (stan == STAN_KONIEC) sprintf(nazwaStanu, "KONIEC");
143
144     LCD_ClearScreen();
145
146     char linia1[17];
147     char linia2[17];
148
149     sprintf(linia1, "MOC:%3d%% %02d:%02d", mocProcent, minuty,
150         sekundy);
151     sprintf(linia2, "STAN: %-10s", nazwaStanu);
152
153     sprintf(buforLcd, "%-16s%-16s", linia1, linia2);
154
155     LCD_PutString(buforLcd, 32);
156 }

```

```

156     __delay32(400000);
157 }
158
159 return 0;
160 }

```

6 Zadanie 5: Zegar szachowy

6.1 Krótki opis zadania

Celem zadania jest stworzenie prostego zegara szachowego, który naprzemiennie odlicza czas dla dwóch graczy. Bazowy czas ustawiony jest na 5min. Po wciśnięciu przycisku (S3) gracz 1 uruchamia zegar gracza 2 i zaczyna grę. Następnie gracz 2 po zakończeniu swojego ruchu używa przycisku (S6) aby wznowić od poprzedniego stanu odliczanie czasu przeciwnika. Gdy któremuś z graczy skończy się czas, wyświetlana jest o tym informacja na ekranie LCD. Można użyć przycisku (S4) aby zresetować grę i przywrócić obu graczom początkowe 5min.

6.2 Kod realizujący zadanie

Listing 5: Zegar szachowy

```

1  #include <xc.h>
2  #include <libpic30.h>
3  #include <stdbool.h>
4  #include <stdint.h>
5  #include <stdio.h>
6  #include "buttons.h"
7  #include "lcd.h"
8
9  //stany zegara szachowego
10 typedef enum {
11     STAN_OCZEKIWANIE,
12     STAN_GRACZ1_MYSLI,
13     STAN_GRACZ2_MYSLI,
14     STAN_KONIEC_G1_PRZEGRAL,
15     STAN_KONIEC_G2_PRZEGRAL
16 } StanZegara;
17
18 int main(void) {
19     AD1PCFG = 0xFFFF;
20     TRISA = 0x0080;
21     LATA = 0x0000;
22
23     LCD_Initialize();
24     LCD_ClearScreen();
25
26     StanZegara stan = STAN_OCZEKIWANIE;
27     StanZegara staryStan = STAN_OCZEKIWANIE;
28
29     long czasG1 = 3000;
30     long czasG2 = 3000;
31
32     long staryCzasG1 = -1;

```

```

33     long staryCzasG2 = -1;
34
35     bool s3Poprzednio = false;
36     bool s6Poprzednio = false;
37     bool s4Poprzednio = false;
38
39     char buforLcd[33];
40
41     while(1) {
42         //odczyt przyciskow graczy
43         bool s3Teraz = BUTTON_IsPressed(BUTTON_S3); // Gracz 1
44         bool s6Teraz = BUTTON_IsPressed(BUTTON_S6); // Gracz 2
45         bool s4Teraz = BUTTON_IsPressed(BUTTON_S4); // Reset
46
47         bool wcisnietoG1 = s3Teraz && !s3Poprzednio;
48         bool wcisnietoG2 = s6Teraz && !s6Poprzednio;
49         bool wcisnietoReset = s4Teraz && !s4Poprzednio;
50
51         s3Poprzednio = s3Teraz;
52         s6Poprzednio = s6Teraz;
53         s4Poprzednio = s4Teraz;
54
55         //logika stanow zegara
56         if (wcisnietoReset) {
57             stan = STAN_OCZEKIWANIE;
58             czasG1 = 3000;
59             czasG2 = 3000;
60             LCD_ClearScreen();
61         }
62
63         if (stan == STAN_OCZEKIWANIE) {
64             //pierwsze wcisniecie startuje zegar przeciwnika
65             if (wcisnietoG1) stan = STAN_GRACZ2_MYSLI;
66             else if (wcisnietoG2) stan = STAN_GRACZ1_MYSLI;
67         }
68         else if (stan == STAN_GRACZ1_MYSLI) {
69             if (wcisnietoG1) {
70                 stan = STAN_GRACZ2_MYSLI;
71             } else {
72                 czasG1--;
73                 if (czasG1 <= 0) {
74                     czasG1 = 0;
75                     stan = STAN_KONIEC_G1_PRZEGRAL;
76                     LCD_ClearScreen();
77                 }
78             }
79         }
80         else if (stan == STAN_GRACZ2_MYSLI) {
81             if (wcisnietoG2) {
82                 stan = STAN_GRACZ1_MYSLI;
83             } else {
84                 czasG2--; // Zegar gracza 2 tyka (co 100ms)
85                 if (czasG2 <= 0) {
86                     czasG2 = 0;
87                     stan = STAN_KONIEC_G2_PRZEGRAL;
88                     LCD_ClearScreen();
89                 }
90             }

```

```

91     }
92
93     //wyswietlacz lcd
94     if (stan != staryStan || (czasG1/10) != (staryCzasG1/10) || (
95         czasG2/10) != (staryCzasG2/10)) {
96         staryStan = stan;
97         staryCzasG1 = czasG1;
98         staryCzasG2 = czasG2;
99
100        if (stan == STAN_KONIEC_G1_PRZEGRAL) {
101            sprintf(buforLcd, "%-16s%-16s", "CZAS MINAL!", "GRACZ 1
102                PRZEGRAL");
103            LCD_PutString(buforLcd, 32);
104        }
105        else if (stan == STAN_KONIEC_G2_PRZEGRAL) {
106            sprintf(buforLcd, "%-16s%-16s", "CZAS MINAL!", "GRACZ 2
107                PRZEGRAL");
108            LCD_PutString(buforLcd, 32);
109        }
110        else {
111            //przeliczanie dziesiatych czesci sekundy na format MM:
112                SS
113            int sekundyG1 = czasG1 / 10;
114            int minG1 = sekundyG1 / 60;
115            int secG1 = sekundyG1 % 60;
116
117            int sekundyG2 = czasG2 / 10;
118            int minG2 = sekundyG2 / 60;
119            int secG2 = sekundyG2 % 60;
120
121            //strzałka ktora pokazuje na gracza ktory myśli
122            char* aktywnyG1 = (stan == STAN_GRACZ1_MYSLI) ? "< MYSLI
123                " : " ";
124            char* aktywnyG2 = (stan == STAN_GRACZ2_MYSLI) ? "< MYSLI
125                " : " ";
126            if (stan == STAN_OCZEKIWANIE) {
127                aktywnyG1 = " GOTOWY";
128                aktywnyG2 = " GOTOWY";
129            }
130
131            char linia1[17];
132            char linia2[17];
133
134            sprintf(linia1, "G1: %02d:%02d%7s", minG1, secG1,
135                aktywnyG1);
136            sprintf(linia2, "G2: %02d:%02d%7s", minG2, secG2,
137                aktywnyG2);
138
139            sprintf(buforLcd, "%-16s%-16s", linia1, linia2);
140
141            LCD_ClearScreen();
142            LCD_PutString(buforLcd, 32);
143        }
144    }
145
146    __delay32(400000);
147}
148return 0;

```

7 Problemy i trudności

W trakcie implementacji powyższych programów napotkano kilka pomniejszych trudności programistycznych oraz sprzętowych:

- **Integer Overflow:** W zadaniu z kuchenką mikrofalową standardowe operacje matematyczne (np. `adcVal * 100`) na rejestrach 16-bitowych powodowały overflow. Konieczne było dopisanie `100L`, wymuszającego operację 32-bitową.

8 Źródła

1. Przekazane przez prowadzącego pliki nagłówkowe `lcd.h`, `adc.h` oraz `buttons.h`.
2. Materiały z przedmiotu Systemy Wbudowane.
3. Oficjalna dokumentacja kompilatora XC16.